Problem Set 6

CHEM 5541 - Chemical Dynamics

Samuel Arnold

10/15/26

1.) Langevvin Model for Ion-Molecule Collisions

$$\mathrm{Ar}^+ + \mathrm{N}_2 \xrightarrow{k_r} \mathrm{N}_2^+ + \mathrm{Ar}$$

1a.)

$$V_{eff}(r) = \frac{L^2}{2\mu r^2} - \frac{\alpha e^2}{2(4\pi\epsilon_0)^2 r^4}$$

```
using Plots

gr()

b = 10e-10
ϵ_0 = 8.85e-12
α = 1.76e-30 * (4 * pi * ϵ_0)
m_Ar = 39.944
m_N2 = 28.016
μ = ((m_Ar * m_N2) / (m_Ar + m_N2)) / (6.022e+26)
k_B = 1.380649e-23
T = 300
v_rel = sqrt((8 * k_B * T) / (pi * μ))
L = μ * v_rel * b
e = 1.602176634e-19

r_range = range(1e-10, 20e-10, length=1000)     # Angstroms
r_range_plot = range(1,20, length = 1000)

V_eff_J = ((L^2) ./ (2 .* μ .* r_range.^2)) - ((α * e^2) ./ (2 .* (4 * pi * ϵ_0
```

```
V_eff_meV = V_eff_J * 6.242e21


r_star = (e / ((4 * pi * ϵ_0) * L)) * sqrt(2 * μ * α)
V_star = ((L^2) / (2 * μ * r_star^2)) - ((α * e^2) / (2 * (4 * pi * ϵ_0)^2 * r_

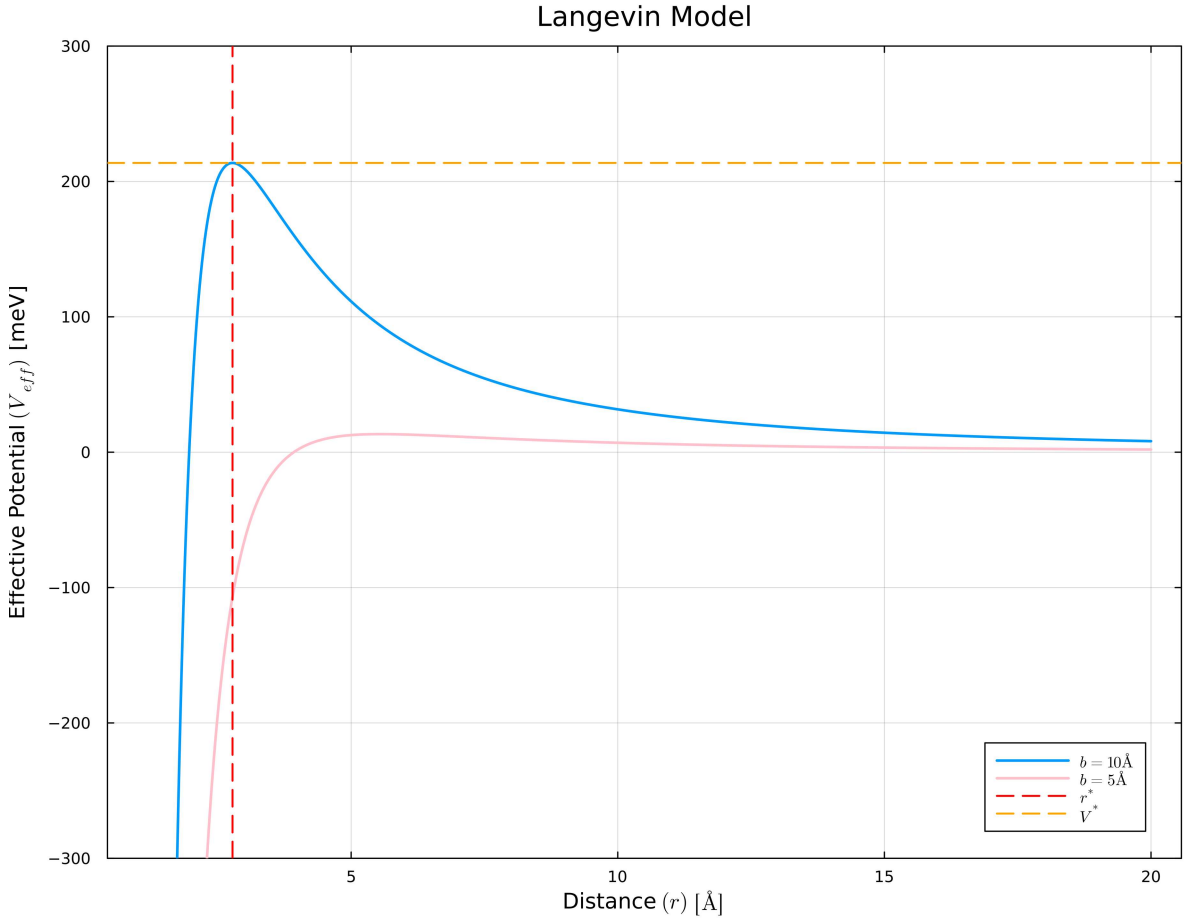

plot(
    r_range_plot,
    V_eff_meV,
    title="Langevin Model",
    xlabel=raw"Distance $(r)$ $[\AA]$",
    ylabel=raw"Effective Potential $(V_{eff})$ [meV]",
    label=raw"$b = 10 \AA$",
    ylims = (-300,300),
    framestyle=:box,
    linewidth=2.0,
    size = (900, 700),
    dpi = 1000,
)

b2 = 5e-10
L_5 = μ * v_rel * b2
V_eff_J_5 = ((L_5^2) ./ (2 .* μ .* r_range.^2)) - ((α * e^2) ./ (2 .* (4 * pi *
V_eff_meV_5 = V_eff_J_5 * 6.242e21

plot!(
    r_range_plot,
    V_eff_meV_5,
    label=raw"$b = 5 \AA$",
    linewidth=2.0,
    color=:pink,
)

vline!(
    [r_star] * 1e10,
    label=raw"$r^*$",
    linestyle=:dash,
    linewidth=1.5,
    color=:red,
)

hline!(
    [V_star] * 6.242e21,
    label=raw"$V^*$",
    linestyle=:dash,
    linewidth=1.5,
    color=:orange,
```



1b.)

$$E = \frac{1}{2}\mu \backslash \text{lang} v_{rel} \backslash \text{rang}^2$$

```
E = (1/2) * μ * v_rel^2 * 6.242e21
println("The energy with a speed at 300 K is: ", round(E, digits=3), " meV")
```

```
The energy with a speed at 300 K is: 32.918 meV
```

This is not enough energy to get over the barrier, but with $b = 5 \backslash \text{AA}$, there is sufficient energy.

1c.)

$$b^* = \left[\frac{1}{(4\pi\epsilon_0)^2} \frac{2\alpha e^2}{E} \right]^{1/4}$$

$$\sigma_L = \pi b^{*2} \frac{\pi e}{4\pi\epsilon_0} \sqrt{\frac{2\alpha}{E}}$$

$$k_L = 2\pi \left(\frac{e}{4\pi\epsilon_0} \right) \sqrt{\frac{\alpha}{\mu}}$$

```
b_star = (((1) / (4 * pi * ϵ_0)^2) * ((2 * α * e^2) / (E / 6.242e21)))^(1/4) *
println("The critical impact parameter at 300 K is: ", round(b_star, digits=3),

σ_L = pi * (b_star / 1e10)^2 * 1e20
println("The Langevin cross section at 300 K is: ", round(σ_L, digits=3), " ang

k_L = 2 * pi * ((e) / (4 * pi * ϵ_0)) * sqrt(α / μ) * 1e6
println("The Langevin rate constant at 300 K is: ", round(k_L, digits=11), " cm
```

```
The critical impact parameter at 300 K is: 6.265 angstroms
The Langevin cross section at 300 K is: 123.31 angstroms^2
The Langevin rate constant at 300 K is: 7.7e-10 cm^3 /s
```

2.)

:$
V*M(r) = D[1-e^{-\alpha(r-re)}]^2$

2a.)

$$|\alpha \Delta r| << 1$$

$$e^{-\alpha(r-r_e)} \approx 1 - \alpha(r - r_e) + \frac{1}{2}[\alpha(r - r_e)]^2 - \dots$$

$$1 - e^{-\alpha(r-r_e)} \approx 1 - [1 - \alpha(r - r_e)] = \alpha(r - r_e)$$

$$V_M(r) \approx D[\alpha(r - r_e)]^2 = D\alpha^2(r - r_e)^2$$

$$\frac{1}{2}K = D\alpha^2$$

$$K = 2D\alpha^2$$

$$\omega_0 = \sqrt{\frac{K}{\mu}} = \alpha\sqrt{\frac{2D}{\mu}}$$

```
using Plots

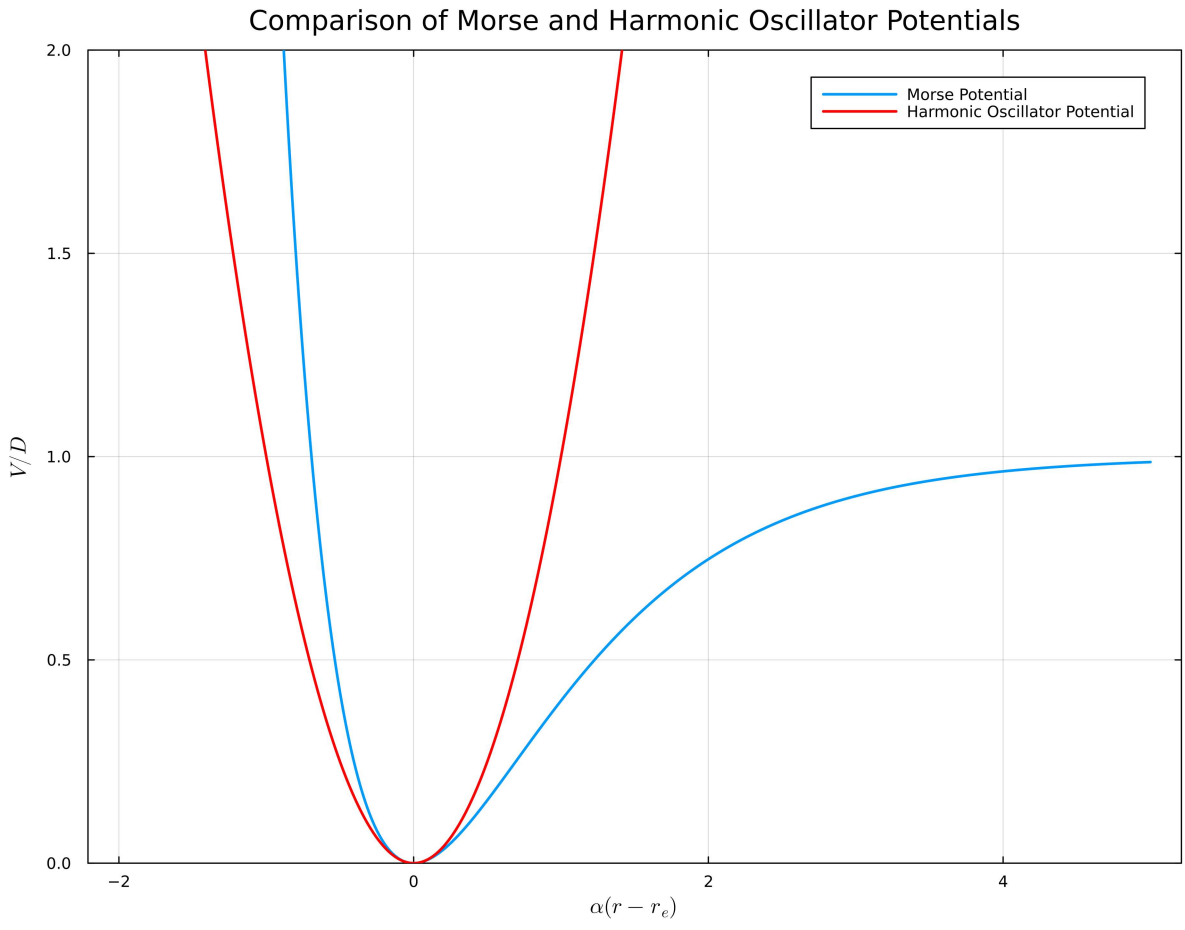
gr()

range_norm = LinRange(-2, 5, 1000)
```

```
V_M_norm = (1 .- exp.( .- range_norm)).^2
V_HO_norm = (range_norm).^2

plot(
    range_norm,
    V_M_norm,
    title="Comparison of Morse and Harmonic Oscillator Potentials",
    xlabel=raw"$\alpha(r-r_e)$",
    ylabel=raw"$V/D$",
    label="Morse Potential",
    ylims = (0,2),
    framestyle=:box,
    linewidth=2.0,
    size = (900, 700),
    dpi = 1000,
)

plot!(
    range_norm,
    V_HO_norm,
    label="Harmonic Oscillator Potential",
    linewidth=2.0,
    color=:red,
)
```



2b.)

$$E_v = \hbar\omega_0\left(v+\frac{1}{2}\right) - \frac{\hbar^2\omega_0^2}{4D}\left(v+\frac{1}{2}\right)^2$$

$$\frac{dE_n}{dv} = \hbar\omega_0 - \frac{\hbar^2\omega_0^2}{2D}\left(v_{max}+\frac{1}{2}\right) = 0$$

$$1 - \frac{\hbar\omega_0}{2D}\left(v_{max}+\frac{1}{2}\right) = 0$$

$$v_{max}+\frac{1}{2} = \frac{2D}{\hbar\omega_0}$$

$$v_{max} = \frac{2D}{\hbar\omega_0} - \frac{1}{2}$$

$$\Delta v = \pm 1$$

$$v_{max+1} = \frac{2D}{\hbar\omega_0} + \frac{1}{2}$$

For the harmonic oscillator, the energy levels are:

$$E_n = \hbar\omega_0\left(n+\frac{1}{2}\right)$$

if $E \leq D$

$$\hbar\omega_0\left(n+\frac{1}{2}\right) \leq D$$

$$n \leq \frac{D}{\hbar\omega_0} - \frac{1}{2}$$

3.) Runge-Kutte Numerical Integration

```
using CSV
using DataFrames
using Plots

function kinetics_simulation()
    A_0 = 1.0         # mol/L
    B_0 = 0.0      # mol/L
    k_1 = 1          # s^-1
    k_2 = 0.1          # s^-1
    delta_t = 0.05     # s
```

```
T = 5           # s


t_array = Float64[]
A_array = Float64[]
B_array = Float64[]

A = A_0
B = B_0
t = 0.0

open("S.Arnold_PS6_Q3_Simple_output.txt", "w") do io
    write(io, "# Parameters: A_0=$A_0 mol/L, B_0=$B_0 mol/L, k_1=$k_1 s^-1,
    write(io, "Time(s)\t[A](mol/L)\t[B](mol/L)\n")

    while t <= T
        push!(t_array, t)
        push!(A_array, A)
        push!(B_array, B)

        write(io, "$t\t$A\t$B\n")

        # Rate laws
        dA = (-k_1 * A + k_2 * B) * delta_t
        dB = (k_1 * A - k_2 * B) * delta_t

        A += dA
        B += dB
        t += delta_t
    end
end

df = CSV.read("S.Arnold_PS6_Q3_Simple_output.txt", DataFrame, delim='\t', c
print(df)

Plots.plot(
    t_array,
    A_array,
    title="Simple First-Order Kinetics Simulation",
    label="[A]",
    xlabel="Time [s]",
    ylabel="Concentration [mol/L]",
    color=:blue,
    linewidth=2,
    size = (900, 700),
    dpi = 1000,
)

plot!(
    t_array,
```

```
        B_array,
        label="[B]",
        color=:red,
        linewidth=2
    )
end

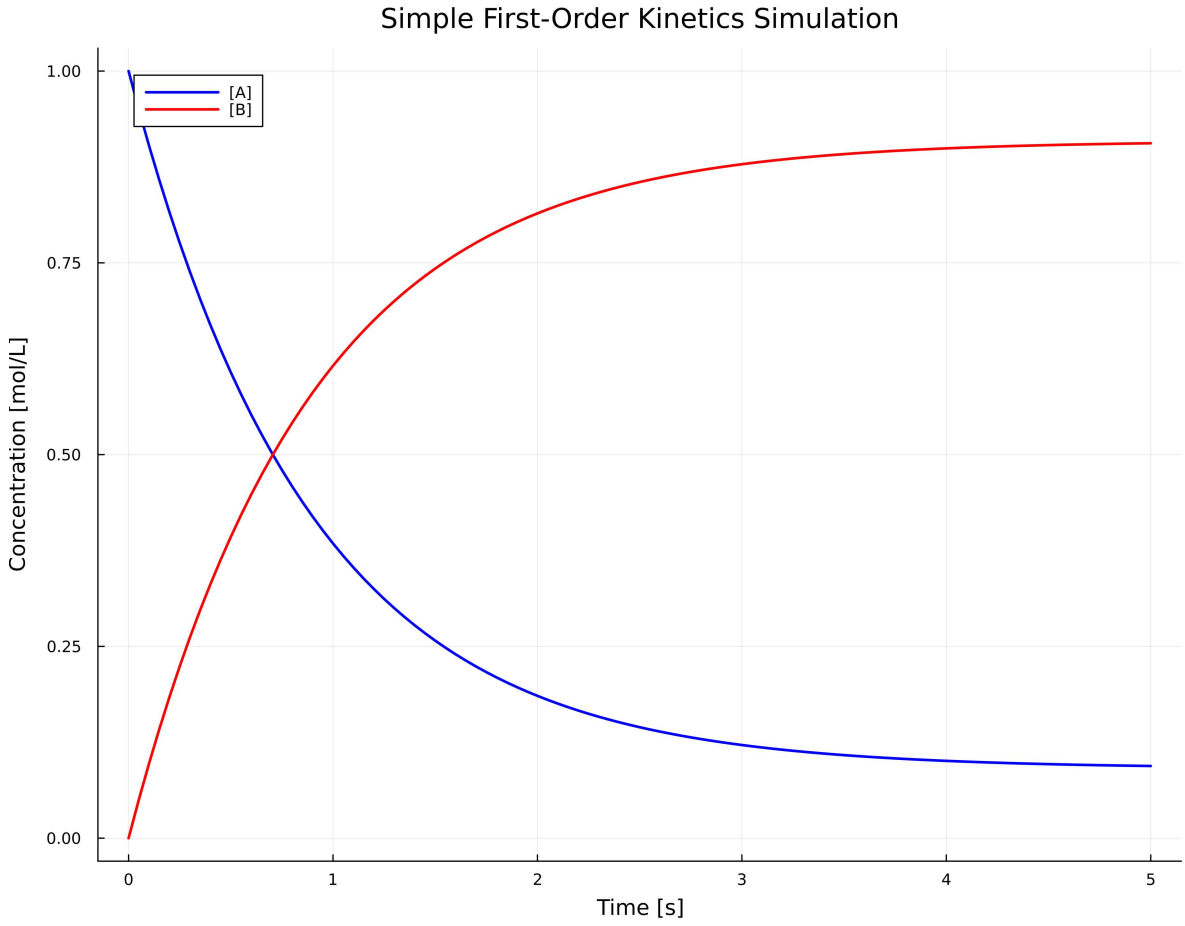
kinetics_simulation()
```

101×3 DataFrame

Row	Time(s)	[A](mol/L)	[B](mol/L)
	Float64	Float64	Float64
1	0.0	1.0	0.0
2	0.05	0.95	0.05
3	0.1	0.90275	0.09725
4	0.15	0.858099	0.141901
5	0.2	0.815903	0.184097
6	0.25	0.776029	0.223971
7	0.3	0.738347	0.261653
8	0.35	0.702738	0.297262
9	0.4	0.669087	0.330913
10	0.45	0.637288	0.362712
11	0.5	0.607237	0.392763
12	0.55	0.578839	0.421161
13	0.6	0.552003	0.447997
14	0.65	0.526642	0.473358
15	0.7	0.502677	0.497323
16	0.75	0.48003	0.51997
17	0.8	0.458628	0.541372
18	0.85	0.438404	0.561596
19	0.9	0.419291	0.580709
20	0.95	0.40123	0.59877
21	1.0	0.384163	0.615837
22	1.05	0.368034	0.631966
23	1.1	0.352792	0.647208
24	1.15	0.338388	0.661612
25	1.2	0.324777	0.675223
26	1.25	0.311914	0.688086
27	1.3	0.299759	0.700241
28	1.35	0.288272	0.711728
29	1.4	0.277417	0.722583
30	1.45	0.267159	0.732841
31	1.5	0.257466	0.742534
32	1.55	0.248305	0.751695
33	1.6	0.239648	0.760352
34	1.65	0.231468	0.768532
35	1.7	0.223737	0.776263
36	1.75	0.216431	0.783569

```
37 |    1.8    0.209528     0.790472
38 |    1.85   0.203004     0.796996
39 |    1.9    0.196838     0.803162
40 |    1.95   0.191012     0.808988
41 |    2.0    0.185507     0.814493
42 |    2.05   0.180304     0.819696
43 |    2.1    0.175387     0.824613
44 |    2.15   0.170741     0.829259
45 |    2.2    0.16635      0.83365
46 |    2.25   0.162201     0.837799
47 |    2.3    0.15828      0.84172
48 |    2.35   0.154574     0.845426
49 |    2.4    0.151073     0.848927
50 |    2.45   0.147764     0.852236
51 |    2.5    0.144637     0.855363
52 |    2.55   0.141682     0.858318
53 |    2.6    0.138889     0.861111
54 |    2.65   0.13625      0.86375
55 |    2.7    0.133757     0.866243
56 |    2.75   0.1314       0.8686
57 |    2.8    0.129173     0.870827
58 |    2.85   0.127068     0.872932
59 |    2.9    0.12508      0.87492
60 |    2.95   0.1232       0.8768
61 |    3.0    0.121424     0.878576
62 |    3.05   0.119746     0.880254
63 |    3.1    0.11816      0.88184
64 |    3.15   0.116661     0.883339
65 |    3.2    0.115245     0.884755
66 |    3.25   0.113906     0.886094
67 |    3.3    0.112641     0.887359
68 |    3.35   0.111446     0.888554
69 |    3.4    0.110317     0.889683
70 |    3.45   0.109249     0.890751
71 |    3.5    0.108241     0.891759
72 |    3.55   0.107287     0.892713
73 |    3.6    0.106386     0.893614
74 |    3.65   0.105535     0.894465
75 |    3.7    0.104731     0.895269
76 |    3.75   0.103971     0.896029
77 |    3.8    0.103252     0.896748
78 |    3.85   0.102573     0.897427
79 |    3.9    0.101932     0.898068
80 |    3.95   0.101326     0.898674
81 |    4.0    0.100753     0.899247
82 |    4.05   0.100211     0.899789
83 |    4.1    0.0996996    0.9003
84 |    4.15   0.0992162    0.900784
85 |    4.2    0.0987593    0.901241
86 |    4.25   0.0983275    0.901672
```

87	4.3	0.0979195	0.902081
88	4.35	0.0975339	0.902466
89	4.4	0.0971696	0.90283
90	4.45	0.0968252	0.903175
91	4.5	0.0964998	0.9035
92	4.55	0.0961923	0.903808
93	4.6	0.0959018	0.904098
94	4.65	0.0956272	0.904373
95	4.7	0.0953677	0.904632
96	4.75	0.0951225	0.904878
97	4.8	0.0948907	0.905109
98	4.85	0.0946717	0.905328
99	4.9	0.0944648	0.905535
100	4.95	0.0942692	0.905731
101	5.0	0.0940844	0.905916




```
using CSV
using DataFrames
using Plots

function kinetics_simulation()
    A_0 = 1.0         # mol/L
    B_0 = 0.0      # mol/L
    k_1 = 1          # s^-1
    k_2 = 0.1          # s^-1
    delta_t = 0.05     # s
```

```
T = 5           # s


t_array = Float64[]
A_array = Float64[]
B_array = Float64[]

A = A_0
B = B_0
t = 0.0

f_A(A, B) = -k_1 * A + k_2 * B
f_B(A, B) =  k_1 * A - k_2 * B

open("S.Arnold_PS6_Q3_RK4_output.txt", "w") do io
    write(io, "# Parameters: A_0=$A_0 mol/L, B_0=$B_0 mol/L, k_1=$k_1 s^-1,
    write(io, "Time(s)\t[A](mol/L)\t[B](mol/L)\n")

    while t <= T
        push!(t_array, t)
        push!(A_array, A)
        push!(B_array, B)

        write(io, "$t\t$A\t$B\n")

        A_k_1 = f_A(A,B)
        B_k_1 = f_B(A,B)

        A_k_2 = f_A(A + A_k_1 * (delta_t/2), B + B_k_1 * (delta_t/2))
        B_k_2 = f_B(A + A_k_1 * (delta_t/2), B + B_k_1 * (delta_t/2))

        A_k_3 = f_A(A + A_k_2 * (delta_t/2), B + B_k_2 * (delta_t/2))
        B_k_3 = f_B(A + A_k_2 * (delta_t/2), B + B_k_2 * (delta_t/2))

        A_k_4 = f_A(A + A_k_3 * delta_t, B + B_k_3 * delta_t)
        B_k_4 = f_B(A + A_k_3 * delta_t, B + B_k_3 * delta_t)

        A += (delta_t/6) * (A_k_1 + 2*A_k_2 + 2*A_k_3 + A_k_4)
        B += (delta_t/6) * (B_k_1 + 2*B_k_2 + 2*B_k_3 + B_k_4)
        t += delta_t
    end
end

df = CSV.read("S.Arnold_PS6_Q3_RK4_output.txt", DataFrame, delim='\t', comm
print(df)

Plots.plot(
    t_array,
    A_array,
    title="RK4 First-Order Kinetics Simulation",
```

```
        label="[A]",
        xlabel="Time [s]",
        ylabel="Concentration [mol/L]",
        color=:blue,
        linewidth=2,
        size = (900, 700),
        dpi = 1000,
    )

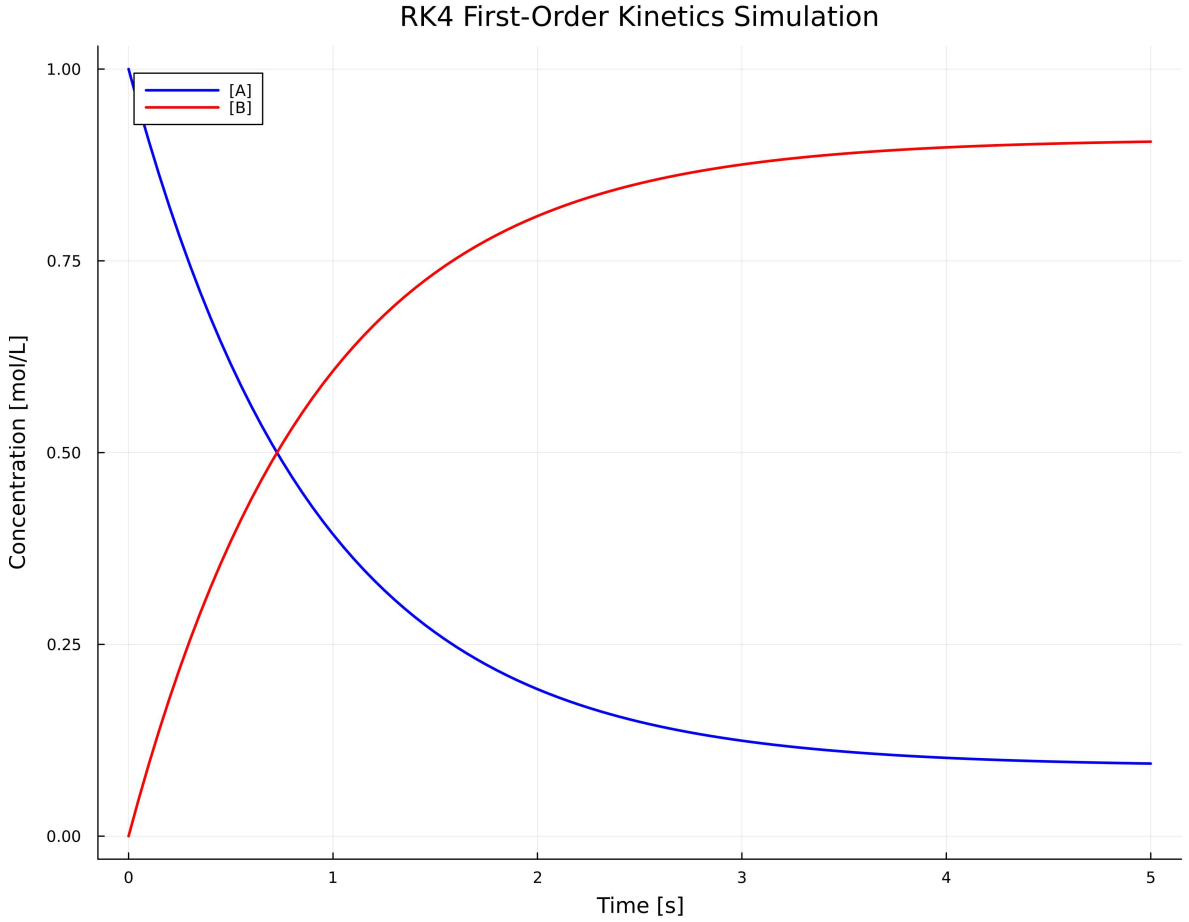
    plot!(
        t_array,
        B_array,
        label="[B]",
        color=:red,
        linewidth=2
    )
end

kinetics_simulation()
```

```
101×3 DataFrame
 Row │ Time(s)  [A](mol/L)  [B](mol/L)
     │ Float64  Float64     Float64
─────┼──────────────────────────────────
   1 │    0.0   1.0         0.0
   2 │    0.05  0.95135     0.0486499
   3 │    0.1   0.905304    0.0946962
   4 │    0.15  0.861722    0.138278
   5 │    0.2   0.820472    0.179528
   6 │    0.25  0.781429    0.218571
   7 │    0.3   0.744476    0.255524
   8 │    0.35  0.709501    0.290499
   9 │    0.4   0.676397    0.323603
  10 │    0.45  0.645064    0.354936
  11 │    0.5   0.615409    0.384591
  12 │    0.55  0.58734     0.41266
  13 │    0.6   0.560774    0.439226
  14 │    0.65  0.535629    0.464371
  15 │    0.7   0.51183     0.48817
  16 │    0.75  0.489305    0.510695
  17 │    0.8   0.467984    0.532016
  18 │    0.85  0.447805    0.552195
  19 │    0.9   0.428706    0.571294
  20 │    0.95  0.410629    0.589371
  21 │    1.0   0.393519    0.606481
  22 │    1.05  0.377325    0.622675
  23 │    1.1   0.361998    0.638002
  24 │    1.15  0.34749     0.65251
  25 │    1.2   0.333759    0.666241
```

```
26 |    1.25   0.320763    0.679237
27 |    1.3    0.308463    0.691537
28 |    1.35   0.29682     0.70318
29 |    1.4    0.285801    0.714199
30 |    1.45   0.275371    0.724629
31 |    1.5    0.2655      0.7345
32 |    1.55   0.256157    0.743843
33 |    1.6    0.247314    0.752686
34 |    1.65   0.238944    0.761056
35 |    1.7    0.231022    0.768978
36 |    1.75   0.223523    0.776477
37 |    1.8    0.216427    0.783573
38 |    1.85   0.20971     0.79029
39 |    1.9    0.203352    0.796648
40 |    1.95   0.197335    0.802665
41 |    2.0    0.191639    0.808361
42 |    2.05   0.186249    0.813751
43 |    2.1    0.181147    0.818853
44 |    2.15   0.176318    0.823682
45 |    2.2    0.171747    0.828253
46 |    2.25   0.167421    0.832579
47 |    2.3    0.163326    0.836674
48 |    2.35   0.159451    0.840549
49 |    2.4    0.155783    0.844217
50 |    2.45   0.152311    0.847689
51 |    2.5    0.149025    0.850975
52 |    2.55   0.145915    0.854085
53 |    2.6    0.142972    0.857028
54 |    2.65   0.140185    0.859815
55 |    2.7    0.137548    0.862452
56 |    2.75   0.135053    0.864947
57 |    2.8    0.13269     0.86731
58 |    2.85   0.130454    0.869546
59 |    2.9    0.128338    0.871662
60 |    2.95   0.126335    0.873665
61 |    3.0    0.124439    0.875561
62 |    3.05   0.122645    0.877355
63 |    3.1    0.120947    0.879053
64 |    3.15   0.119339    0.880661
65 |    3.2    0.117818    0.882182
66 |    3.25   0.116378    0.883622
67 |    3.3    0.115015    0.884985
68 |    3.35   0.113725    0.886275
69 |    3.4    0.112504    0.887496
70 |    3.45   0.111348    0.888652
71 |    3.5    0.110254    0.889746
72 |    3.55   0.109219    0.890781
73 |    3.6    0.108239    0.891761
74 |    3.65   0.107312    0.892688
75 |    3.7    0.106434    0.893566
```

76	3.75	0.105603	0.894397
77	3.8	0.104817	0.895183
78	3.85	0.104073	0.895927
79	3.9	0.103368	0.896632
80	3.95	0.102701	0.897299
81	4.0	0.10207	0.89793
82	4.05	0.101473	0.898527
83	4.1	0.100908	0.899092
84	4.15	0.100373	0.899627
85	4.2	0.0998662	0.900134
86	4.25	0.0993868	0.900613
87	4.3	0.0989332	0.901067
88	4.35	0.0985038	0.901496
89	4.4	0.0980973	0.901903
90	4.45	0.0977126	0.902287
91	4.5	0.0973486	0.902651
92	4.55	0.0970039	0.902996
93	4.6	0.0966778	0.903322
94	4.65	0.0963691	0.903631
95	4.7	0.0960769	0.903923
96	4.75	0.0958003	0.9042
97	4.8	0.0955386	0.904461
98	4.85	0.0952908	0.904709
99	4.9	0.0950563	0.904944
100	4.95	0.0948344	0.905166
101	5.0	0.0946243	0.905376



For the exact solution:

$$[A](t) = [A_0]\left(1 - \frac{k_1}{k_1 + k_2}[1 - e^{-(k_1+k_2)t}]\right)$$

```
using CSV
using DataFrames
using Plots

function kinetics_simulation()
    A_0 = 1.0         # mol/L
    B_0 = 0.0      # mol/L
    k_1 = 1          # s^-1
    k_2 = 0.1          # s^-1
    delta_t = 0.05      # s
    T = 5           # s


    t_array = Float64[]
    dev_simple = Float64[]
    dev_rk4 = Float64[]

    A_simple, A_rk4 = A_0, A_0
    B_simple, B_rk4 = B_0, B_0
    t = 0.0
```

```
f_A(A, B) = -k_1 * A + k_2 * B
f_B(A, B) =  k_1 * A - k_2 * B


open("S.Arnold_PS6_Q3_Exact_output.txt", "w") do io
    write(io, "# Parameters: A_0=$A_0 mol/L, B_0=$B_0 mol/L, k_1=$k_1 s^-1,
    write(io, "Time(s)\t[A](mol/L)\n")

    while t <= T
        push!(t_array, t)

        A_exact = A_0 * (1 - (k_1 / (k_1 + k_2)) * (1 - exp(-(k_1 + k_2) *t

        write(io, "$t\t$A_exact\n")

        push!(dev_simple, A_simple - A_exact)
        push!(dev_rk4, A_rk4 - A_exact)

        dA = (-k_1 * A_simple + k_2 * B_simple) * delta_t
        dB = (k_1 * A_simple - k_2 * B_simple) * delta_t

        A_simple += dA
        B_simple += dB


        A_k_1 = f_A(A_rk4,B_rk4)
        B_k_1 = f_B(A_rk4,B_rk4)

        A_k_2 = f_A(A_rk4 + A_k_1 * (delta_t/2), B_rk4 + B_k_1 * (delta_t/2
        B_k_2 = f_B(A_rk4 + A_k_1 * (delta_t/2), B_rk4 + B_k_1 * (delta_t/2

        A_k_3 = f_A(A_rk4 + A_k_2 * (delta_t/2), B_rk4 + B_k_2 * (delta_t/2
        B_k_3 = f_B(A_rk4 + A_k_2 * (delta_t/2), B_rk4 + B_k_2 * (delta_t/2

        A_k_4 = f_A(A_rk4 + A_k_3 * delta_t, B_rk4 + B_k_3 * delta_t)
        B_k_4 = f_B(A_rk4 + A_k_3 * delta_t, B_rk4 + B_k_3 * delta_t)

        A_rk4 += (delta_t/6) * (A_k_1 + 2*A_k_2 + 2*A_k_3 + A_k_4)
        B_rk4 += (delta_t/6) * (B_k_1 + 2*B_k_2 + 2*B_k_3 + B_k_4)



        t += delta_t
    end
end

df = CSV.read("S.Arnold_PS6_Q3_Exact_output.txt", DataFrame, delim='\t', co
print(df)

Plots.plot(
```

```
        t_array,
        dev_simple,
        title="Simulation Deviations (Simple vs RK4)",
        label="Simple",
        xlabel="Time [s]",
        ylabel=raw"Deviation from $[A]_{exact}$",
        color=:blue,
        linewidth=2,
        size = (900, 700),
        dpi = 1000,
    )

    plot!(
        t_array,
        dev_rk4,
        label="RK4",
        color=:red,
        linewidth=2
    )

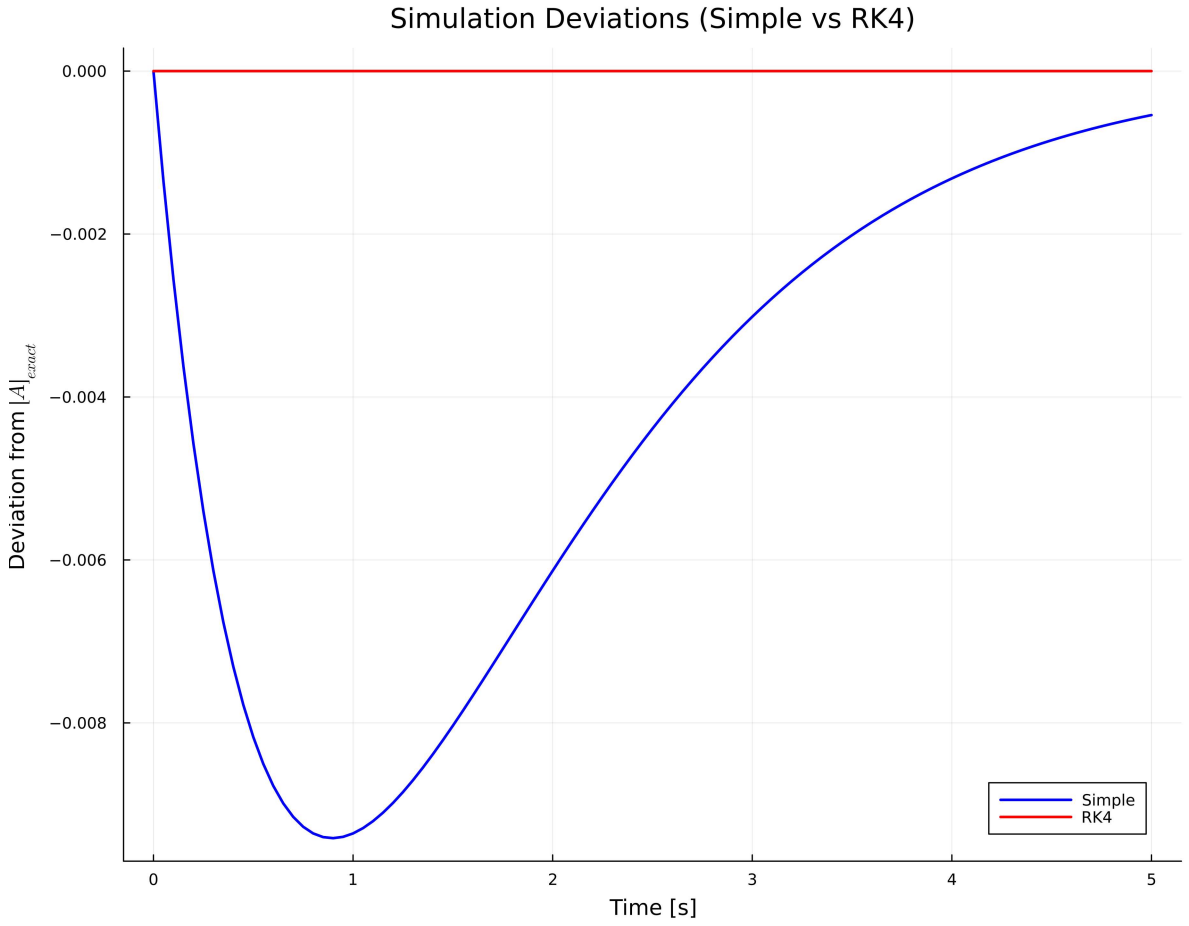
end

kinetics_simulation()
```

```
101×2 DataFrame
 Row │ Time(s)  [A](mol/L)
     │ Float64  Float64
─────┼──────────────────────
   1 │    0.0    1.0
   2 │    0.05   0.95135
   3 │    0.1    0.905304
   4 │    0.15   0.861722
   5 │    0.2    0.820472
   6 │    0.25   0.781429
   7 │    0.3    0.744476
   8 │    0.35   0.709501
   9 │    0.4    0.676397
  10 │    0.45   0.645064
  11 │    0.5    0.615409
  12 │    0.55   0.58734
  13 │    0.6    0.560774
  14 │    0.65   0.535629
  15 │    0.7    0.51183
  16 │    0.75   0.489305
  17 │    0.8    0.467984
  18 │    0.85   0.447805
  19 │    0.9    0.428706
  20 │    0.95   0.410629
  21 │    1.0    0.393519
```

```
22 |    1.05   0.377325
23 |    1.1    0.361998
24 |    1.15   0.34749
25 |    1.2    0.333759
26 |    1.25   0.320763
27 |    1.3    0.308463
28 |    1.35   0.29682
29 |    1.4    0.285801
30 |    1.45   0.275371
31 |    1.5    0.2655
32 |    1.55   0.256157
33 |    1.6    0.247314
34 |    1.65   0.238944
35 |    1.7    0.231022
36 |    1.75   0.223523
37 |    1.8    0.216427
38 |    1.85   0.20971
39 |    1.9    0.203352
40 |    1.95   0.197335
41 |    2.0    0.191639
42 |    2.05   0.186249
43 |    2.1    0.181147
44 |    2.15   0.176318
45 |    2.2    0.171747
46 |    2.25   0.167421
47 |    2.3    0.163326
48 |    2.35   0.159451
49 |    2.4    0.155783
50 |    2.45   0.152311
51 |    2.5    0.149025
52 |    2.55   0.145915
53 |    2.6    0.142972
54 |    2.65   0.140185
55 |    2.7    0.137548
56 |    2.75   0.135053
57 |    2.8    0.13269
58 |    2.85   0.130454
59 |    2.9    0.128338
60 |    2.95   0.126335
61 |    3.0    0.124439
62 |    3.05   0.122645
63 |    3.1    0.120947
64 |    3.15   0.119339
65 |    3.2    0.117818
66 |    3.25   0.116378
67 |    3.3    0.115015
68 |    3.35   0.113725
69 |    3.4    0.112504
70 |    3.45   0.111348
71 |    3.5    0.110254
```

```
 72 |    3.55   0.109219
 73 |    3.6    0.108239
 74 |    3.65   0.107312
 75 |    3.7    0.106434
 76 |    3.75   0.105603
 77 |    3.8    0.104817
 78 |    3.85   0.104073
 79 |    3.9    0.103368
 80 |    3.95   0.102701
 81 |    4.0    0.10207
 82 |    4.05   0.101473
 83 |    4.1    0.100908
 84 |    4.15   0.100373
 85 |    4.2    0.0998662
 86 |    4.25   0.0993868
 87 |    4.3    0.0989332
 88 |    4.35   0.0985037
 89 |    4.4    0.0980973
 90 |    4.45   0.0977126
 91 |    4.5    0.0973486
 92 |    4.55   0.0970039
 93 |    4.6    0.0966778
 94 |    4.65   0.0963691
 95 |    4.7    0.0960769
 96 |    4.75   0.0958003
 97 |    4.8    0.0955386
 98 |    4.85   0.0952908
 99 |    4.9    0.0950563
100 |    4.95   0.0948344
101 |    5.0    0.0946243
```



Published from S.Arnold - Problem Set 6 - CHEM 5541.ipynb using Weave.jl v0.10.12 on 2026-10-16.